

int len (A, s, e) {A, s, e} length of LPS in sub string from

if (s == e) ret 1;
if (s > e) ret 0; // s to e in A.

if (s > e) ret 0;

if (A[s] == A[e])

ret 2 + LPS(A, s+1, e-1);

if (A[s] == A[e])

else,

ret max(LPS(A, s+1, e), LPS(A, s, e-1));

else,

ret max(LPS(A, s, N+1), LPS(A, s+1, e));

s: a g b d b a

$$DP[i][j] = \begin{cases} 2 + DP[i+1][j-1] & \text{if } S[i] == S[j] \\ \max(DP[i+1][j], DP[i][j-1]) & \text{else} \end{cases}$$

	0	1	2	3	4	5
	a	g	b	d	b	a
0	a	1	1	1	3	5
1	g	0	1	1	3	3
2	b	0	0	1	3	3
3	d	0	0	0	1	1
4	b	0	0	0	0	1
5	a	0	0	0	0	1

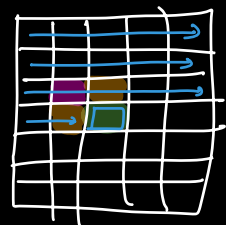
- Define the function/
state of DP

$DP[i][j] \rightarrow$

- Dimensions $\rightarrow$ no of unique
states

- Direction of iteration

$DP[i][j]$ depends on $DP[i-1][j-1]$, $DP[i][j-1]$, $DP[i-1][j]$



Knap Sack

0/1

Given N items each with a weight & a value.
Find the maximum value that can be obtained by
picking items such that total weight of all items
being picked $\leq$ max capacity of sack (K)

- Every item can be picked at max once.
- Taking an item partially is not allowed.

Ex $N=4$ $K=50$

$w[]$	: 20	, 10	, 30	, 40	→ 220
$v[]$	: 100	60	120	150	

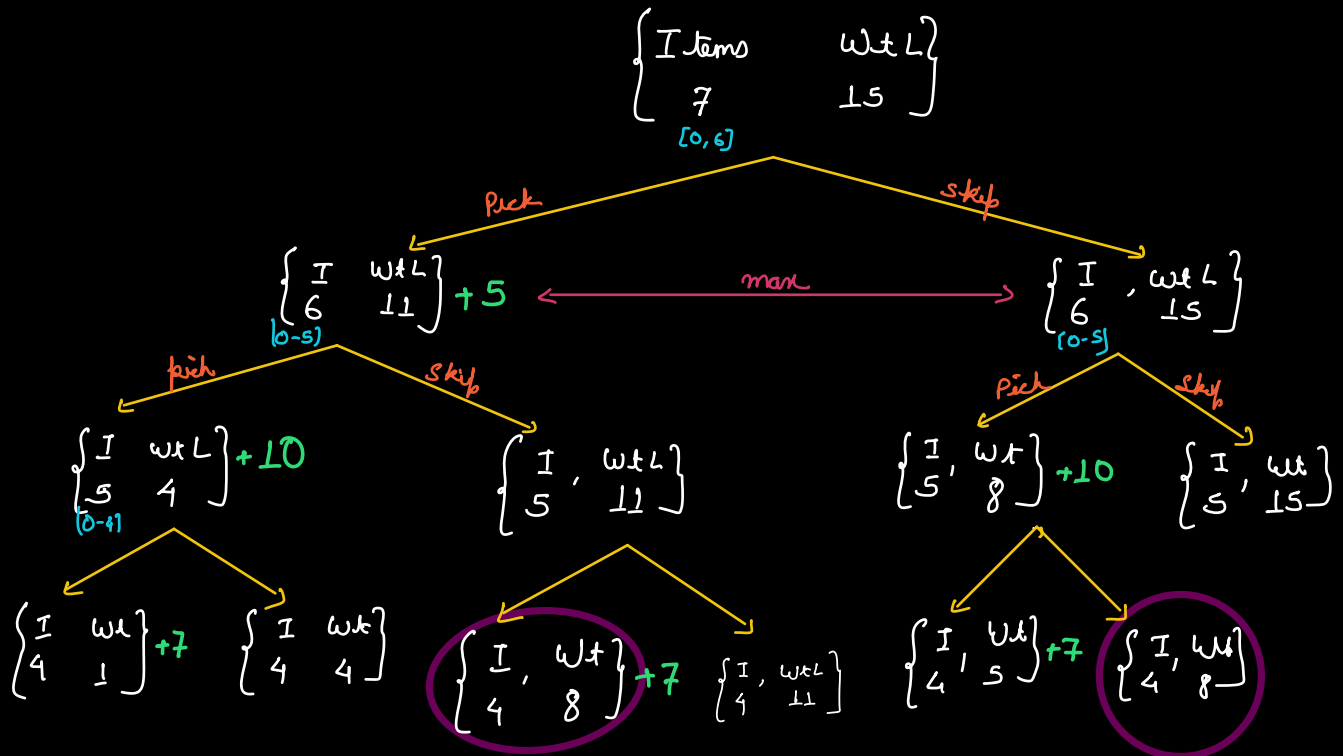
Approach 1 Start picking elements in order of values

w	: 40	10	→ 210
v	: 150	60	

$w[]$	: 20	, 10	, 30	, 40
$v[]$	: 100	60	120	150
v_{per}	: 5	6	4	3.75
unit wt				

w	: 10	20	→ 160
v	: 60	100	

$N: 7$ $K = 15$
 $w: 4, 1, 5, 4, 3, 7, 4$
 $v: 3, 2, 8, 3, 7, 10, \underline{5}$



```
int knapSack (w[], v[], N, K) {
```

// Return max value that can be created from the
 first N items $[0, (N-1)]$ within cut limits of K
 if $(N == 0 \parallel K == 0)$

return 0;

// skip the item

```
int ans = knapSack (w, v, N-1, K);
```

// pick the item

```
if (w[N-1] <= K) {
```

```
    ans = max (ans,
```

```
                knapSack (w, v, N-1, K - w[N-1]) + v[N-1],
```

```
    )
```

```
return ans;
```

```
}
```

1

$$j \longrightarrow [0, N] \Rightarrow N+1 \longrightarrow (N+1) \times (K+1)$$
$$j \longrightarrow [0, K] \Rightarrow K+1$$

Corrent wet

$DP[i][j] \rightarrow$ store the max value obtained from first i items $[0, i-1]$ within a wt limit of j .

TC: $O(NK)$ let limit

↑

Size of $w[]$ & $v[]$ array

 $O(QN)$
$$O(N) + O(Q)$$

$\left[\begin{array}{c} \text{Pseudo-Polynomial} \\ T_C \end{array} \right]$

	0	1	2	3	4
w:	3	6	5	2	4
v:	12	20	15	6	10

$$K=8$$

sc: $O(NK)$

	0	1	2	3	4	5	6	7	8
0	0	0	0	0	0	0	0	0	0
1	0	0	0	12	12	12	12	12	12
2	0	0	0	12	12	12	20	20	20
3	0	0	0	12	12	15	20	20	27
4	0	0	6	12	12	18	20	21	27
5	0	0	6	12	12	18	20	22	27

$$DP[i][j] = \begin{cases} DP[i-1][j] & // skip \\ DP[i-1][j - w[i-1]] \\ \quad + \\ \quad V[i-1] \end{cases}$$

$DP[3][5] \rightarrow 15$

OR

$$\underline{6} + \underline{DP[3][2]} \rightarrow 18 \rightarrow$$

Coin Change Problem

Q. Given N coins & a sum K .

Find no. of ways to form the sum K if one coin can be picked at most once.

$C : [7, 4, 9, 6, 11, 5, 13, 8]$ $K = 22$

KnapSack

Items $\rightarrow$ Coins

wt limit $\rightarrow$ Sum K .

if $K = 0 \rightarrow 1$

if $N = 0 \rightarrow 0$

0/∞ Unbounded KnapSack

Given N items each with a weight & a value.

Find the maximum value that can be obtained by picking items such that total weight of all items being picked $\leq$ max capacity of sack (K)

- Every item can be picked as many times as we want.
- Taking an item partially is not allowed.

$$DP[i][j] = \max \begin{cases} DP[i-1][j] & // \text{skip} \\ DP[i][j - w[i-1]] + v[i-1] & // \text{Pick} \\ & (\text{if } j \geq w[i-1]) \end{cases}$$

Wine Profit